

HQ Connection: Configuration Audit and Plan

Rating: 2.8 / 10. The mechanism works for one machine and does not reach a fleet.

Repository: cirqen-labs, branch improvement-plan | Reviewed: 19 September 2026 Ninth in the review set.
Extends 05_HQ_AND_SYNC_AUDIT.pdf, which covered what HQ does. This one covers how a desktop finds HQ.

1. The short answer

You asked what happens if you change the host. I tested it rather than reasoning about it, using the real `CirqenConfig` class against a scratch data directory.

Step	<code>sync.api_url</code> on the machine
Machine installed today; first run writes <code>config.json</code>	old host
You change the default in <code>config.py</code> and ship a new build	old host (unchanged)
Ops sets <code>SYNC_API_URL</code> and <code>HQ_SERVER_URL</code> on that machine	old host (ignored)
A brand-new install after your change	new host

Three consequences follow.

- Only new installs pick up a new host.** On first run `_load_from_json` writes the *entire* default config, addresses included, into `config.json`. From then on the stored copy wins over the default, so every installed desktop is pinned to the address it was born with.
- Environment variables cannot fix it in production.** Settings loads with `use_env_file=False`, and in that mode the environment is consulted only for four secrets (`SECRET_ENV`). The URL variables are honoured in development and silently ignored in the packaged app.
- The only remedy today is editing `config.json` by hand on every desktop.** `provisioning.json` cannot help: it carries secrets only, not addresses.

If the old host is switched off, field work continues, because writes are local first. But nothing syncs, and certificates stay in `approved_pending_certificate` because HQ is the only thing that mints their numbers (see document 05, section 3).

So the hardcoding is worse than "the URL is in a Python file". The URL is *copied into user data* at install time, and nothing later can correct the copy.

2. What is hardcoded, and where

Four different HQ addresses appear in the code. Two are live, one is the database, and one belongs to a database that was already decommissioned.

- `hq-server-dgs6.onrender.com`, the sync API and certificate authority. Written in `config.py`, `sync/config.py`, `sync/agent_prelude.py` (twice), `sync/mirror.py` and `build.py` (three times).
- `cirqen-hq.onrender.com`, the update server. Written in `config.py`, `sync/config.py`, `bulider_tools/app_updates.py` and `bulider_tools/ui.py`, plus docstrings.

- **The Supabase pooler host and user**, the HQ database, which every desktop reaches **directly**. Written in `config.py`, `sync/config.py` and `helper_scripts/patch_config.py`.
- `dpg-...ohio-postgres.render.com`, the **old** HQ database, already migrated away from. Still written in `sync/agent_prelude.py`, `sync/sync_agent_5.py` and `build.py`.

The last row is evidence, not speculation. A comment in `sync/sync_agent_1.py` records that the agent was "still pointing at an old/decommissioned Render host after `hq_db` was migrated to Supabase". A host change has already happened once and left stale copies behind. Three of them are still there. `sync_agent_5` (the missing-parent fetcher) will fall back to the dead host, with the old database name and user, whenever `POSTGRES_HQ_HOST` is unset.

The two live services (sync and updates) are genuinely different hosts, so a move of one does not move the other. The code treats them as unrelated settings with no shared source.

I checked this against the HQ repository (`~/Desktop/hq_server`, Render service `cirqen-hq-api`, started as `gunicorn og_server:app`). It serves the sync and certificate API only. It contains `hq_update_server.py` and `update_api.py`, but `og_server` imports neither, so `/api/updates/*` is served only by the separate update service from `cirqen-labs/hq_server`. That also means the `.../api/updates` address that `build.py` writes onto the sync host (finding C-9) points at a path that host does not serve.

3. Findings

#	Severity	Finding
C-1	High	Existing installs are pinned to their first-run address. Demonstrated in section 1.
C-2	High	The production loader ignores every non-secret environment variable, so there is no supported override on an installed machine.
C-3	High	Update signature checking is off as the repository stands. <code>UPDATE_PUBLIC_KEY = ""</code> in <code>updates/updater.py</code> , and <code>settings.UPDATE_SYSTEM</code> never sets <code>public_key</code> . The log says "unsigned mode". Whoever controls the update host controls code that runs on every desktop, so an abandoned or hijacked hostname becomes remote code execution across the fleet. This turns a host <i>move</i> from an inconvenience into a security event.
C-4	High	HQ API keys do not survive a new host. Keys are stored hashed in HQ's <code>sync_api_keys</code> table, and the enrollment code is cleared from <code>config.json</code> after first use (<code>test_enrollment.py</code> asserts it). A new HQ without that table rejects every desktop, and no desktop holds a code to re-enroll.
C-5	Medium	<code>config.py</code> and <code>sync/config.py</code> are two tracked copies of the same class and have already drifted . <code>sync/config.py</code> has the dynamic-port fix; <code>config.py</code> does not. Django imports the first (<code>Equiper/settings.py</code>), the builder imports the second (<code>bulider_tools/runtime.py</code>), and both write the same <code>config.json</code> .
C-6	Medium	The builder's connection indicator ignores the configured address. <code>bulider_tools/ui.py</code> looks for <code>hq_server_url</code> , <code>server_url</code> and <code>hq_url</code> at the top level of <code>config.json</code> , but the real value lives at <code>update.server_url</code> . The lookup always misses, so the indicator pings the hardcoded fallback. After a move it would report the state of the <i>old</i> server. <code>app_updates.py</code> does look under <code>update</code> , so the two builder components disagree.
C-7	Medium	Every desktop connects straight to HQ's Postgres (mirror, <code>cert_conflict_guard</code> , <code>sync_agent_5</code> , Django's HQ alias). A move of the database therefore needs a change, and a distributed password, on every machine. This is the deepest coupling.

#	Severity	Finding
C-8	Medium	Failure is silent. Nothing validates that a URL is well formed, uses https, or has the expected /api/sync suffix. validate_config only checks that update.server_url is non-empty. Stale fallbacks (C-2, section 2) turn a missing setting into a quiet connection to the wrong place rather than an error.
C-9	Low	build.py writes .update_config.json into packages with a third address (.../api/updates on the sync host). Nothing reads it. It is dead configuration that will mislead the next person.
C-10	Low	The installer template sync_agent.env.example is generated from the old Render database, and bulider_tools/services.py logs a hardcoded "polling https://cirqen-hq.onrender.com" regardless of configuration.

Adjacent, and not about hosts: the default local_db password is a literal in config.py and ships in every installer, and the legacy .env fallback in agent_prelude.py embeds a database host and a password. Neither belongs in source. Both should be rotated and removed as part of Phase 0.

4. What already works

Stated because the plan builds on it.

There is a real configuration layer. CirqenConfig merges defaults, config.json and, for secrets, environment and provisioning.json, with a documented precedence. The plan extends it rather than replacing it.

Secrets were handled properly. No HQ credential is baked into source (apart from the local password noted above). SECURITY.md and test_config_secrets.py cover it.

The sync agent already reads from config. sync_agent_1 builds the mirror from the loaded config instead of raw os.getenv calls, and says why. That is the pattern to apply everywhere.

Health checking tolerates cold starts. check_hq_online retries on 502, 503, 504 and timeouts, but not on a hard connection error. Moving to a new host does not require redesigning it.

Field work never depends on HQ. The write model is local first, so a bad or unreachable address degrades sync, not the site.

5. Rating

Nine dimensions, weighted by how much each matters when the goal is "the app is distributed to many desktops and HQ can move".

Dimension	Weight	Score	Why
Change reaches installed machines	25%	1	Demonstrated failure. No remote mechanism.
Single source of truth	15%	3	Four hostnames, over a dozen live literals, two drifting copies of the config class.
Components agree with each other	15%	3	UI ignores config, builder and Django read different keys, fallbacks point at dead hosts.
Override without editing code	10%	4	Works in development; in production only by hand-editing JSON per machine.

Dimension	Weight	Score	Why
Validation and failure behaviour	10%	4	Health check is good; nothing validates URLs and fallbacks fail silently.
Safety of a host move (trust, keys)	10%	3	Unsigned updates and non-portable keys.
Secrets handling	5%	5	Good layering; local password and legacy fallback remain in source.
Documentation and runbook	5%	3	Rotation is documented; moving HQ is not.
Tests	5%	2	Consumers are tested with an injected URL. No test proves config reaches them.

Weighted total: 2.8 / 10.

The score is low because the heaviest dimension is the one the current design cannot satisfy at all. The configuration *layer* is worth about a 6 on its own. It is let down by where the values are stored and by the leftovers around it. The path to a high score is short because the layer exists.

6. The plan

Six phases, ordered so that each one is useful alone and none depends on a decision you have not made. Effort figures are estimates for one developer who already knows the code.

Phase 0. Stop the bleeding (about 1 day)

Small, safe, and needed before anything else so the later work has one thing to edit.

1. Remove every dead-host fallback: `agent_prelude.py` (legacy loader and `is_online` default), `sync_agent_5.py`, `sync/mirror.py` CLI defaults, and the `build.py` env template. Where a value is required and absent, raise a clear error instead of defaulting.
2. Make `sync/config.py` the single implementation and turn the top-level `config.py` into a one-line re-export (or the reverse). Port the dynamic-port fix so nothing is lost. Confirm the PyInstaller spec still finds it.
3. Fix `ui.py` to read `update.server_url` (through the config class, not raw JSON) and make `app_updates.py` use the same call.
4. Delete `.update_config.json` generation, or make it read the resolved value.
5. Remove the local database password literal and the legacy fallback credential from source, and rotate both.

Done when: `grep -rE "onrender\.com|supabase\.com|ohio-postgres" --include=*.py` finds nothing outside a single defaults module.

Phase 1. One resolver, one precedence (about 1.5 days)

Add `core/hq_endpoints.py` (or a section of the config class) exposing the only functions any other code may use:

```
sync_url()      -> "https://<host>/api/sync"
update_url()    -> "https://<host>"
health_url()    -> sync_url() + "/health"
hq_db_dsn()     -> resolved database settings
describe()      -> each value plus where it came from (env / config.json / default)
```

Precedence for **every** setting, secret or not: environment, then `config.json`, then `provisioning.json`, then built-in default. This closes C-2.

Validation lives here and runs at startup: `https` required except for `localhost`, no trailing slash, no doubled `/api/sync`, host resolvable-looking. A bad value fails loudly with the setting name and its source. This closes C-8.

Point every consumer at the resolver: the sync agent modules, `mirror`, `core/hq_link.py`, the updates app, the `bulider_tools` package and `settings.py`.

Done when: changing the value in one place changes it in all of them, proved by test (below).

Phase 2. Make a change reach installed machines (about 2 to 3 days)

This is the phase that moves the score, because it fixes the heaviest dimension.

1. **Stop persisting defaults.** Write only values that differ from the built-in default (or that an operator set) into `config.json`. A default then stays a default, and changing it reaches everyone on the next update.
2. **One-time migration for machines already pinned.** Ship a list of known old defaults (`LEGACY_DEFAULTS`). On load, a stored value equal to a legacy default is treated as unset. Values an operator genuinely edited are left alone.
3. **Remote redirect.** HQ serves a small document at `/api/sync/endpoints` containing the current sync and update addresses, a version and a signature. Each desktop fetches it at heartbeat time, verifies the Ed25519 signature against a public key baked into the build, and stores the result as an *override layer* (above the default, below the environment). A move then becomes: publish the new document from the old host, wait for the fleet to pick it up, retire the old host. Section 9 describes the admin page that publishes it.
4. **Ordered fallbacks.** Allow `endpoints.fallbacks`, tried after N consecutive failures of the primary. This covers the case where the old host dies before a desktop has fetched the redirect.

Done when: the simulation in section 1 ends with the new host on the existing install.

Phase 3. Make a move safe (about 1 to 2 days)

1. Generate the update signing key and set `UPDATE_PUBLIC_KEY` in the build. Refuse unsigned packages. **Do this before Phase 2 ships**, because the redirect in Phase 2 is only as trustworthy as the signature check. This closes C-3.
2. Make keys portable. The runbook (Phase 5) requires copying `sync_api_keys` to the new HQ before cutover. Add an admin "migration code" that lets a desktop with a known `client_id` re-enroll without the original one-time code. This closes C-4.

Phase 4. Take the database off the desktop (4 to 6 days; needs your decision)

Route the few direct HQ Postgres uses (`mirror`, `cert_conflict_guard`, `sync_agent_5`, the Django HQ alias) through HQ API endpoints. Afterwards a desktop knows one address, no database host or password leaves HQ, and C-7 closes.

This is the largest item and changes a working data path, so it is optional for the goal you stated. Phases 0 to 3 make the *URLs* flexible; only Phase 4 makes the *database* movable without touching desktops. If HQ's database will stay on Supabase for the foreseeable future, defer it.

Phase 5. Operate it (about 1 day)

1. `python manage.py hq_endpoint --show | --set sync=<url> | --test`. `--show` prints each value and its source, `--test` calls the health endpoint and reports the result.

2. A read-only "Connection" panel in the desktop app showing the same, with a Test button. Editing is deliberately not offered on the desktop: fleet settings are changed once, on HQ, through the page in section 9.
3. A "Moving HQ" runbook in SECURITY.md: copy keys, publish the redirect, watch the fleet's heartbeats reach the new host, keep the old host answering for a set period, then retire it.

Tests that hold the result in place

- Precedence: environment beats config.json beats provisioning beats default, for URLs as well as secrets.
- **Fleet simulation:** the section 1 experiment as a permanent test. An existing config.json written by an old build must pick up a changed default.
- **Consumer contract:** for each component, set the resolver to a sentinel host and assert the component's outgoing request uses it.
- **Literal guard:** a test that fails if a HQ hostname appears in source outside the defaults module. This is what stops the next Render-to-Supabase move from leaving stale copies.
- Redirect: an unsigned or wrongly signed endpoints document is ignored.

7. Sequence and outcome

Order	Phase	Effort	Score after
1	0. Remove stale fallbacks, unify the config class	1 day	4.5
2	1. Resolver, precedence, validation, tests	1.5 days	6.0
3	3.1 Turn on update signing	0.5 day	6.5
4	2. Reach installed machines, redirect, fallbacks	2 to 3 days	8.0
5	3.2 Portable keys, 5. Tooling and runbook	2 days	8.5
6	4. Database off the desktop	4 to 6 days	9.0

The scores after each phase are projections from the same weights as section 5, not measurements. The remaining points after Phase 5 are the direct database coupling, which is why Phase 4 is what moves it past 8.5.

8. Decisions I need from you

1. **One HQ host or two?** Today sync and updates are separate services. If you intend to merge them, the resolver takes a single base address; if not, it keeps two.
2. **How many desktops are already installed, and can you reach them?** If the count is small and reachable, Phase 2 step 2 (the migration) is enough. If not, the remote redirect is essential.
3. **Will the old host stay alive during a move?** The redirect design depends on it. If the old host may vanish without notice, the fallback list carries the load and needs to ship first.
4. **Does the update signing key already exist?** If it has been generated, Phase 3.1 is a config change. If not, it needs a key ceremony and a place to keep the private half.
5. **Is Phase 4 in scope?** It is the only phase that changes a working data path.

9. A settings page that reaches every desktop

> **Superseded in part by section 17.** This section assumed the moving host announces its own > replacement. The update server is a permanently fixed address, which is a better anchor and > removes the requirement that the old host stay alive. Read section 17 first; the page design, > guardrails and signing trade-off below still stand.

Yes. It has to live on **HQ**, not in the desktop app, and it has to work over connections the *desktop* opens. Hospital and workshop machines sit behind firewalls and NAT, so HQ cannot open a connection to them. Two such connections already exist. `sync_agent_7` sends a heartbeat every 60 seconds, and HQ's `/api/sync/heartbeat` replies with a JSON body that the desktop currently ignores. Each desktop also holds open a Server-Sent Events stream to `/api/sync/events`, which HQ can write to at any time. The heartbeat reply is the reliable channel; the stream is a fast one on top of it.

9.1 How a change travels

1. You edit the settings on the HQ admin page and press Publish. HQ stores them as a new numbered version and signs the document.
2. Each heartbeat reply now carries `config_version`. A desktop that sees a version newer than its own downloads the signed document.
3. The desktop checks the signature, confirms the new address is reachable, and only then adopts it.
4. The desktop reports back on its next heartbeat: adopted, or failed and why.

HQ can also send a `config` event down each open stream so connected desktops pick the change up within seconds instead of within a minute. **That stream cannot be the only channel.** It is capped at `SSE_MAX_CLIENTS` (24 by default) because each stream pins one of the server's 32 worker threads, and streams are recycled every 600 seconds. Beyond 24 desktops, some will not hold a stream, so the heartbeat has to carry the version for everyone. An offline desktop catches up on reconnect either way. The page can therefore show "47 desktops: 41 adopted, 4 pending, 2 unseen for over a week", with the names of the stragglers, which is what a manual fix could never tell you.

9.2 What the page contains

- **Settings:** sync address, update address, ordered fallback addresses, and later the database host. Each shows its current value and who set it.
- **Test:** before anything is published, HQ calls the new address's health endpoint, checks its TLS certificate, and confirms it accepts a probe key. A failed test blocks Publish.
- **Publish to:** one named test desktop, a percentage of the fleet, or everyone. The update server already supports a rollout percentage (`_in_rollout` in `hq_server/main.py`), so the same idea is reused.
- **Adoption panel:** per-desktop state and last-seen time, live.
- **History:** every version with who published it and when, and a one-click "republish the previous version".
- **Access:** a real admin login and a confirmation step that asks you to type the new hostname. Today HQ has only an admin bearer token (`HQ_ADMIN_TOKEN`) for its API, not a login for a page, so this needs building.

9.3 The guardrails that matter most

A typo in a fleet-wide address would strand every desktop with no way to correct it, so the design assumes mistakes:

1. **Test before publish**, as above.

2. **The desktop probes before adopting.** If the new address does not answer its health check, the desktop keeps the old one and reports the failure.
3. **Automatic revert.** After adopting, if the new address fails for a set period (for example three failed heartbeats over five minutes), the desktop switches back to the previous address and says so. This is the same idea as a router's "confirm within 60 seconds or undo".
4. **Staged rollout**, so a bad address hits one machine first.
5. **Signed, https only, and optionally restricted to an allowed list of hostname patterns**, so a stolen admin session cannot point the fleet at an arbitrary server.
6. **The old host keeps answering** for the agreed period. The redirect is only delivered *by* the old host.

9.4 One trade-off to decide: where the signing key lives

If HQ holds the private key, publishing is one click, but anyone who compromises HQ can redirect the fleet. If the key stays offline on your machine, HQ cannot sign, and each publish needs a signing step. A middle path is recommended: a **separate endpoints key** held by HQ for this document, and the **package-signing key kept offline**. Then a compromised HQ can redirect sync traffic (which the allowed-hostname list narrows) but still cannot make desktops run code, because code packages need the offline key.

9.5 What the page does not do

It does not deliver code. Addresses and intervals are a few hundred bytes and need no restart once every consumer reads them at call time (Phase 1). Code goes through the update pipeline in section 10.

10. How backend updates reach the desktops

This is how the existing pipeline behaves, from reading updates/ and hq_server/.

10.1 The flow today

1. **Publish.** You build a signed package with hq_server/build_package.py and upload it to the update server. Each release can carry a critical flag, a min_version, a rollout_percent, and a yanked kill switch.
2. **Discover.** A Celery beat task, updates.tasks.check_and_apply_updates, asks the update server /api/updates/latest/ every update.check_interval_hours (24 by default). The server answers "no update" for machines outside the rollout window, below min_version, or when the release is yanked. The server also records each check, so it knows every machine's version.
3. **Approve.** Ordinary updates are **not** auto-applied by default (auto_apply_updates is off), so an operator applies them from the Updates page on each desktop. Releases flagged critical **are** auto-applied by default (auto_apply_critical is on).
4. **Preflight and download.** A preflight checks the install directory is writable and there is 500 MB free. The package is then downloaded and its checksum checked. The manifest's Ed25519 signature is verified, then each file's hash.
5. **Protect.** Changed files are backed up, and the local PostgreSQL database is dumped, before anything is touched.
6. **Apply.** Files are written atomically, unchanged files are skipped, deleted files are backed up then removed, and the tree is re-hashed on disk.
7. **Migrate.** Local migrations run on the desktop. The **shared HQ database is migrated once, under a lock** taken from the update server (/api/migrations/acquire/), so fifty desktops updating at once do not collide.

8. **Health check.** `manage.py check` runs in a fresh process against the new code. Any failure in steps 5 to 8 triggers an automatic rollback of files and database.
9. **Restart.** A `.restart_required` marker is written. When the desktop app exits, `bulider_tools/app.py` sees it, stops Django, Celery and the sync agent, and relaunches, so the new backend runs in fresh processes. The code comments record why: without that stop, the old Django process kept running old code while the window looked new.
10. **Report.** The outcome is posted to `/api/updates/report/`, so HQ can see who is on which version.

10.2 The order in which this feature must ship

The redirect client does not exist on installed desktops yet, so the first delivery has to go through this pipeline:

1. Build Phases 0 to 2 into a package. Turn on signing first (Phase 3.1), because that package is the one that teaches every desktop to obey redirects.
2. Publish it with a small `rollout_percent`, watch the machines list, then widen. Flag it `critical` so that desktops with default settings apply it without waiting for an operator.
3. Desktops that have not updated cannot be redirected. `/api/updates/machines/` shows who they are.
4. Only when most of the fleet reports the new version do you make the first real address change from the page.

10.3 Weak points in the current pipeline

These matter because the whole plan depends on updates arriving.

- **Signing is off.** Covered in finding C-3. Turn it on before this is relied on.
- **Slow to converge.** A 24-hour check interval means a release takes at least a day to reach every desktop that is online, and ordinary releases also wait for an operator on each machine. Critical releases skip the wait for approval, not the check interval.
- **One shared key for every desktop.** The update server's `HQ_API_KEY` is a single generated value used by all machines, so it cannot be revoked per desktop the way sync keys can.
- **The update server builds download links from `RENDER_EXTERNAL_URL`.** When the update server moves, that variable must be correct on the new host or desktops will be given bad download links.
- **A restart is needed for code,** and it happens when the desktop app exits. A desktop left running for weeks stays on old code.

10.4 Decisions added by this section

1. **Where does the editable page live?** Recommended: the HQ sync server, which already has the database, the heartbeat and the admin token, and is the one thing every desktop already talks to every minute.
2. **Where does the endpoints signing key live?** Recommended: on HQ, separate from the offline package key, per section 9.4.
3. **Should non-critical updates auto-apply?**

11. What the HQ repository adds to the design

Findings from reading `~/Desktop/hq_server`, and where each piece of section 9 goes.

11.1 Where the code goes

Piece	Location in the HQ repo
Settings and history tables	A new SQL file in migrations/, named like the existing 2026_add_sync_api_keys.sql, run by migration_runner.py
Admin page and publish endpoint	A new route module, imported in the "ROUTE MODULE IMPORTS" block of og_server_core.py beside queue_dashboard
Version in the heartbeat reply	client_heartbeat in certificate_health_client.py. It has two success returns (normal and retry), and both need the field
Instant push to open streams	The SSE code in job_queue_download_update.py
Signed document endpoint	Same new module, served without a client key (see 11.2)
Tests	tests/, which already has a pytest setup and covers API keys, enrollment and alerting

The service runs with `--workers 1`, so a small in-memory cache of the current signed document is safe. If that is ever raised, the cache must move to the database or Redis, because the Procfile warns that a second worker reopens a certificate-number race.

11.2 Prerequisites found in the code

These are read from route decorators and function bodies, not tested against the live server.

- **The existing dashboard has no authentication.** `/api/sync/dashboard` in `queue_dashboard.py` has no `@require_api_key` and no inline check, and `setup_security` installs no global guard. The page lists every client's name, IP address, version and workshop data. The new settings page must not be placed beside it as it stands. It needs its own login, and the dashboard itself should be locked down at the same time.
- **The heartbeat is unauthenticated too.** `/api/sync/heartbeat` also has no decorator or inline check. That is acceptable for a request that only reports status, but it means the `config_version` in its reply must never be trusted on its own. The desktop must always verify the signed document.
- **The signed document should be served without a client key.** It is public by nature and signed. A desktop whose key was lost or rejected (finding C-4) can then still find out where HQ has moved to, which is exactly the desktop that most needs it.
- **Two SSE admin endpoints use the client-carried token.** `/api/sync/events/clients` and `/api/sync/events/evict` compare against `SYNC_AUTH_TOKEN`, the shared token that clients carry, while the recent commit "Per-client sync keys with enrollment; separate admin token" moved the key-admin routes onto `HQ_ADMIN_TOKEN`. These two should move as well.
- **Rate limiting and the IP allow-list exist but the allow-list is off** (`IP_WHITELIST_ENABLED=false` in `render.yaml`). Field sites are unlikely to have fixed IPs, so an allow-list for the admin page would need to cover your office or VPN only.
- **Render dashboard settings override** `render.yaml`. The file's own comments record an outage caused by this. Any new environment variable for the signing key must be set in the dashboard, and the runbook should say so.

11.3 Effect on the plan

Phase 2 and the settings page are unchanged in shape. Two items are added: lock down the dashboard and the two SSE admin endpoints, and add the `config` event to the stream as an accelerator. Neither changes the effort estimates by more than about half a day.

12. Status: addresses and database details (19 September)

The hardcoded addresses and database details (Phases 0 and 1 of section 6, plus the migration half of Phase 2) are done. The full Django suite (563 tests) and the pytest suite (66) pass.

Finding	State
C-1 Installed machines pinned to first-run address	Fixed. Defaults are no longer written to <code>config.json</code> . A file from an older build is migrated once and a copy kept as <code>config.json.pre-endpoints</code> . A changed default now reaches installed machines. Tested, including a machine that skips a release.
C-2 Environment ignored in production	Fixed. <code>SYNC_API_URL</code> , <code>HQ_SERVER_URL</code> and the <code>POSTGRES_HQ_*</code> variables are honoured in the packaged app, and never written to disk.
C-5 Two drifting copies of the config class	Fixed. <code>config.py</code> is the one implementation (with the port fixes that only the second copy had). <code>sync/config.py</code> is a shim that loads it.
C-6 Launcher indicator pinged a hardcoded host	Fixed. <code>ui.py</code> , <code>app_updates.py</code> and <code>services.py</code> resolve the address the same way the updater does.
C-8 No validation	Fixed. <code>validate_config()</code> rejects a bad address by name.
C-9, C-10 Dead config and stale installer template	Fixed. <code>build.py</code> writes addresses from the single block; the template uses <code>HQ_SERVER_URL</code> , which is the variable the code reads (it used <code>HQ_UPDATE_URL</code> , which nothing read).
Old Render database as a fallback	Removed from <code>agent_prelude.py</code> , <code>sync_agent_5.py</code> , <code>mirror.py</code> and <code>build.py</code> . A missing host is now an error, not a guess. <code>sync_agent_5</code> uses the agent's own HQ database settings, including <code>sslmode</code> .
Local database password in source	Removed. A new install generates one; an existing install keeps the one it has. The value is still in git history, so it should be rotated.
Hostnames written in more than one place	Guarded. A test fails if an HQ hostname appears in any Python file other than the defaults block in <code>config.py</code> .

Still open at that point. C-3 (update signing is off), C-4 (keys and enrollment on a new HQ), C-7 (desktops connect straight to the HQ database), the settings page and remote broadcast (sections 9 and 11), and the HQ-side lock-down of the dashboard and stream admin endpoints. The settings page and the command were built the same day — see section 13; the rest stand. The addresses still exist as one block of literals in `config.py`; that block is what a build ships as the fresh-install default.

Two things to know.

1. The HQ database port disagrees between two places: `config.py` ships 5432, while `render.yaml` in the HQ repo and one machine's hand-patched `config.json` use 6543. Both are Supabase pooler modes. The shipped default was left at 5432; whichever is correct for desktops should be set once, in the defaults block.
2. The Django test runner loads settings against the real data directory, so running the tests migrates the developer's own `config.json` in place. This is what happened here. It is harmless (the backup is kept) but it is a side effect of tests that pre-dates this work.

13. Settings and navigation (19 September)

Phase 5 of section 6, brought forward because a machine that cannot be reached by hand needs to be inspectable and correctable by whoever is standing at it.

A page in the application. *Administration* → *HQ Connection* (/settings/hq-connection/), staff only, the same guard the update pages use. It shows every address, **where each one came from**, and whether it can be changed here:

- *shipped default* — follows future releases. This is the state you want on almost every machine.
- *Set on this machine* — pinned, and the page says plainly that a release moving HQ will not move this machine. One button clears every pin.
- *Environment variable* — shown read-only, naming the variable, because the environment wins over anything the page could save.

Leaving a box empty means "follow the shipped default again", so releasing a pin needs no special knowledge.

Test connection probes all three services and reports each one separately: a 401 counts as reachable, since it proves the address is right and only the key is wrong, and a 502/503 is reported as a server waking up rather than as a failure. Saving validates first and writes nothing if any field is bad, so a half-applied form is impossible, and the page says a restart is needed before the sync agent and update checker use the new address.

The same thing without the UI, for headless machines and support calls:

```
python manage.py hq_endpoint --show      # every value and where it came from
python manage.py hq_endpoint --test      # do the configured servers answer?
python manage.py hq_endpoint --set sync.api_url=https://hq.example.com/api/sync
python manage.py hq_endpoint --reset all
```

Both share one module (core/hq_settings.py), so they validate identically. 29 tests cover them; the full Django suite (589) and pytest (66) pass.

A bug this turned up. `setup_environment_variables()` exports every resolved value into the environment, and child processes inherit them. The layering then read those exports back as operator-set variables: sources were mislabelled env, and **every field on the page would have been read-only in the packaged app**. Fixed by recording the exported values and ignoring a variable only while it still holds exactly what we wrote — so a variable an operator really sets is still honoured. Found by running the command rather than by reading the code.

13.1 Rating: can HQ be moved now?

The section 5 dimensions, re-scored.

Dimension	Weight	Was	Now
Change reaches installed machines	25%	1	8
Single source of truth	15%	3	9
Components agree with each other	15%	3	8
Override without editing code	10%	4	9
Validation and failure behaviour	10%	4	8
Safety of a host move (trust, keys)	10%	3	4
Secrets handling	5%	5	8

Dimension	Weight	Was	Now
Documentation and runbook	5%	3	7
Tests	5%	2	9

2.8 → 7.9 / 10. The remaining 2.1 is almost entirely the two unfixed safety items: update signing is still off (C-3) and client keys still do not survive a new HQ (C-4). The first dimension is 8 rather than 10 because a change still travels inside an application release; there is no remote config push yet.

14. The real question: 500 desktops

Addressing is no longer the limit. At 500 machines the limit is HQ itself, and the numbers are not close.

14.1 What one desktop costs HQ

Every desktop runs nine sync threads. Five of them talk to HQ on a timer:

Loop	Interval	Requests/second
Download	5s	0.200
Health check (every 3rd download loop)	15s	0.067
Certificate sync	30s	0.033
Certificate pull	60s	0.017
Heartbeat	60s	0.017
Total per desktop		≈0.33

Plus one permanently open SSE stream, and direct connections to HQ's PostgreSQL.

At 500 desktops: ≈167 requests/second before anyone calibrates anything. That is the idle cost.

14.2 Three walls, in the order they are hit

Wall 1 — the event stream, at roughly 25 desktops. SSE_MAX_CLIENTS is 24 because each stream pins one of the server's 32 threads. The 476 desktops that cannot get a slot do not fail fast: the server waits $\text{SSE_PING_INTERVAL} + 2 = 10$ seconds for a slot before answering 503, and the agent sleeps 10 seconds and reconnects. So every rejected desktop spends about half its life *holding a thread while queuing for a thread*. Offered demand is roughly 240 concurrent waiters against the 8 threads SSE has not already taken. The server stops answering everything, health checks included — which is precisely the outage the Procfile comments describe, except permanent rather than occasional. **This one bites long before 500, and it is the cheapest to fix.**

Wall 2 — database connections, at somewhere around 100–200 desktops. Every desktop connects straight to HQ's Supabase PostgreSQL: Django's hq alias holds a persistent connection (CONN_MAX_AGE 300), and the mirror, the certificate conflict guard and the missing-parent recovery each open their own. Two to three concurrent connections per desktop means **1,000–1,500 connections at 500 desktops**. Supervisor in transaction mode handles a few hundred client connections on typical plans; session mode far fewer. There is no configuration that makes this fit. This is finding C-7, and at this scale it stops being an optional phase.

Wall 3 — the single worker, at around 150–250 requests/second. HQ runs `--workers 1 --threads 32` under the GIL with a 20-connection pool. The obvious fix — more workers — is explicitly forbidden by the Procfile, because `certificate_service_base.py`'s re-entrancy guard is in-process memory and the HTTP certificate path only serialises within one process. **So the certificate-number correctness guard is what caps HQ at one worker, and one worker is what caps the fleet.** Scaling out requires moving that guard to the Redis lock that already exists for the background thread (`og_server_core.cert_processor_lock`). Until then, HQ cannot be scaled horizontally at all.

14.3 What already scales

Worth saying, because three walls is not the whole picture.

- **Rate limiting is keyed per API key, not per IP.** The comment says "supports 500+ devices", and a hospital with forty desktops behind one NAT address gets forty independent limits. Someone thought about this.
- **Writes are local first.** 500 desktops do not stop working when HQ is overloaded; they stop *syncing*. The blast radius of every wall above is delay, not lost work.
- **Update rollout has real controls** — `rollout_percent`, `min_version`, a yanked kill switch, and delta downloads via `from_version`.
- **The heartbeat is the right broadcast channel for a fleet this size.** 500 desktops heartbeating every 60 seconds is 8.3 requests/second, the cheapest traffic on the list, and it reaches every machine within a minute. The settings-page design in section 9 rides on it rather than on the stream, which is what makes it workable at 500 where the stream is not.

14.4 Rating: ready for 500 desktops?

Dimension	Weight	Score	Why
A setting change reaching the fleet	20%	6	Works, but only inside an application release; no remote push yet
HQ request capacity	20%	2	≈167 req/s idle against one worker that cannot be multiplied
HQ database connections	20%	1	1,000+ direct connections; no configuration makes this fit
Update delivery at scale	10%	4	Good controls; untested bandwidth, single instance
Fleet visibility	10%	6	<code>sync_clients</code> and the machines list are genuinely useful; the dashboard has no authentication
Safety when a change is wrong	10%	5	Validation and probes now; no staged rollout or auto-revert for addresses
Per-machine configuration	10%	9	Page, command, environment and provisioning all work

4.2 / 10 for 500 desktops.

The score is dragged down by two dimensions worth 40% between them that no amount of configuration work can lift. It is worth being precise about what that means: **nothing I changed this week is wasted, and nothing I changed this week gets you to 500.** Addressing was the stated problem and it is fixed. Capacity is the actual problem and it is untouched.

14.5 What to do, in order

1. **Fix the SSE rejection (about half a day).** Refuse immediately instead of blocking for 10 seconds, send Retry-After with jitter, and have the agent fall back to interval polling rather than reconnecting every 10 seconds. This alone moves the first wall from ~25 desktops to wherever wall 3 sits, and it is a small, contained change.
2. **Move the certificate guard to the Redis lock, then raise workers.** This is the unlock for everything else; while HQ is pinned at one worker, no other capacity work matters.
3. **Take the database off the desktop (section 6, Phase 4).** Now mandatory rather than optional. Until it is done, the fleet has a hard ceiling in the low hundreds.
4. **Back off and jitter the polling.** Five-second download polling for a field application is aggressive; 30 seconds with instant wake-up from the heartbeat reply would cut idle load about six-fold and cost almost nothing in responsiveness.
5. **Then** build the settings broadcast of sections 9 and 11, on the heartbeat, with staged rollout and auto-revert.

A useful sanity check before any of it: the rate-limit comments claim 500-device support, so someone has sized part of this already. It is worth asking what that was based on, and whether a real load test has ever been run against HQ. Every number in this section is derived from reading the code, not from a load test, and the cheapest way to confirm or refute them is to point 50 simulated agents at a staging HQ and watch the thread pool.

15. Old way against new way

A concrete comparison of the mechanisms, not the scores. Each row is something somebody actually has to do.

Operation	Old way	New way
Move the whole fleet to a new HQ	Impossible without touching every machine. A new default reached new installs only	Change HQ_ENDPOINT_DEFAULTS, ship a build. Every machine follows on next start unless deliberately pinned
Move one site	Edit config.json by hand on each desktop, with no validation and no feedback	Settings page, hq_endpoint --set, an environment variable, or the installer's provisioning.json
Find out what a desktop is using	Read config.json and guess whether it was a default or a choice	The page and --show print the value and which layer supplied it
Discover a typo	Silent. A wrong address looked exactly like an offline HQ	Refused at save time, by name, before anything is written
Check a server answers	No mechanism	Test connection / --test, covering sync, updates and the database separately
Put a machine back on the fleet default	No concept of one	Blank the box, or --reset all; the page says which machines are pinned
Where addresses are written	Four hostnames across more than a dozen files, two of them already stale	One block in config.py; a test fails the build if a hostname appears anywhere else
Environment variables	Honoured in development, silently ignored in the packaged app	Honoured everywhere, never written to disk, shown as read-only on the page
Reaching settings at all	No navigation entry; the updates page needed a typed URL	Administration → System → HQ Connection , and Updates alongside it
Local database password	A literal in config.py, shipped in every installer	Generated per machine on first run

Operation	Old way	New way
Old config.json files	n/a	Migrated once on next start, with a config.json.pre-endpoints copy kept

15.1 What the new parts cost

22 files changed (766 added, 1,330 removed — the deletion is mostly the duplicate config class), 6 new files totalling 1,179 lines including 603 lines of tests. **60 tests** cover the new behaviour; the full Django suite is 594 and pytest 66, all passing, ruff clean.

15.2 Rating the new parts

Part	Rating	Judgement
Endpoint resolution and layering	9	One source, four layers with a stated order, migration for old machines, sources reported. Tested against the exact failure that started this
Validation	8	Catches scheme, suffix, shape and range, names the setting. Does not yet check that a hostname resolves before saving
Settings page	8	Says where each value came from and what is pinned, refuses bad input whole, tests all three services. Needs a restart to take effect, which it states but cannot yet perform
hq_endpoint command	9	Everything the page does, scriptable, correct exit codes. This is what works on 500 machines; the page is for the one in front of you
Removal of stale hosts	9	Dead Render database gone from four files; a guard test stops the next one appearing
Test coverage	9	60 tests, including a fleet-migration simulation and a machine that skips a release
Documentation	7	README section and .env.example; no "Moving HQ" runbook in SECURITY.md yet
Fleet reach	5	A change still travels only inside an application release. No remote push, no staged rollout, no auto-revert
Safety of a move	4	Unchanged: update signing still off, client keys still do not survive a new HQ

Weighted by the section 5 dimensions: 7.9 / 10, from 2.8. The two low rows are the two things deliberately not attempted, and both are already written up: fleet reach in sections 9 and 11, safety in C-3 and C-4.

15.3 What a review pass of this work found

I reviewed my own changes against the codebase's conventions rather than assuming they were right. Three defects, all now fixed and covered by tests:

1. **The navigation entry was in the wrong block.** I put it inside the sidebar's HOD section, but the view is staff-only, so the link appeared only for users who were **both** HOD and staff. Staff administrators without the HOD role could use the page but could not find it. Fixed by giving these pages their own staff-only **System** section, which also surfaces the Updates page — staff-only and previously reachable only by typing the URL. Two regression tests.

2. **Messages rendered twice.** `base.html` already renders Django messages with icons and a dismiss control; my template rendered them again, so every save showed its confirmation twice. The duplicate is removed and the page now uses the shared component.
3. **Three CSS variables did not exist.** I wrote `--bg-input`, `--success`, `--danger`, `--warning` and `--info`; the design system defines `--input-bg`, `--success-color`, `--danger-color`, `--warning-color` and `--info-color`. Every one would have silently fallen back to a hard-coded colour that ignores the dark theme. Now on the real tokens, with a focus ring from `--input-focus-shadow` and `aria-live` on the results area.

A fourth was found earlier, by running the command rather than reading it: `setup_environment_variables()` exports resolved values into the environment and child processes inherit them, so the layer read its own exports back as operator-set variables and **every field would have been read-only in the packaged app**. Fixed by recording exported values and ignoring one only while it still holds exactly what was written.

`coerce_endpoint` was also made public, since the settings page and the command both depend on it; importing a private name across modules was the wrong contract.

The pattern worth noting: all four defects were in the seams — template against design system, view guard against navigation guard, config layer against its own environment exports. None were in the logic, and none would have been caught by the unit tests I had written, because each unit was correct on its own terms.

16. Where this leaves things

Two numbers, because they answer different questions.

- **Can HQ be moved, and can settings be changed without touching each machine?** **7.9 / 10**, from 2.8. Addressing, validation, visibility and per-machine control are done and tested. What remains is fleet reach and the two safety items.
- **Is this ready for 500 desktops?** **4.2 / 10**, unchanged by this work and not improvable by it. Section 14 has the arithmetic: the event stream fails at about 25 desktops, direct database connections at 100–200, and HQ cannot be scaled out at all while the certificate guard keeps it at one worker.

The honest summary is that the configuration problem is solved and the capacity problem is not, and they were never the same problem. The next useful day of work is the SSE rejection fix in section 14.5, which is contained, needs no decisions from you, and moves the first wall from roughly 25 desktops into the low hundreds.

17. Moving the sync HQ from host A to host B

This is the operation the whole document is about, and the architecture makes it easier than section 9 assumed.

17.1 Three hosts, one of them fixed

	What it is	Code	Moves?
Host A → B	Sync API and certificate authority	<code>~/Desktop/hq_server</code> (Flask)	Yes — this is the move
Host C	Update server	<code>cirqen-labs/hq_server</code> (FastAPI)	No — fixed by design
Database	HQ PostgreSQL (Supabase)	—	Possibly, separately

Section 9 had the *old sync host* serve the redirect, which only works while the old host is alive — exactly what you cannot rely on during a move, and useless if host A dies without warning.

A permanently fixed host C removes that problem completely. **The only address that has to be baked into a build is host C's.** Everything else becomes something a desktop asks about.

17.2 How the move works

Host C answers one new question: *where is the sync HQ?*

1. Host C serves GET `/api/endpoints/` — a small signed document naming the current sync API address (and, later, the database).
2. Every desktop polls it on a short interval. This is independent of the sync HQ, so it keeps working when host A is unreachable, overloaded or switched off.
3. The desktop verifies the Ed25519 signature, checks the new address answers `/health`, adopts it, and reports the result on its next check-in.

The move then becomes: stand up host B → change one environment variable on host C → restart it. Every desktop follows within one poll interval. Host A can be switched off as soon as the fleet has moved, and host C tells you when that is.

No release, no visiting desktops, no dependence on the host being retired.

17.3 What has to be built

Less than section 9 implied, and **all of it in this repository** — host C's code is `cirqen-labs/hq_server`, and `~/Desktop/hq_server` needs no changes at all, because it is the thing being moved rather than the thing announcing the move.

- **Host C:** one route that builds the document from its own environment variables and signs it with the Ed25519 key `build_package.py --genkeys` already produces. **No storage** — which matters, see 17.4.
- **Desktop:** a small client that polls, verifies, probes and adopts, writing the result as a layer above the shipped default and below an operator's own pin. The layering built this week already has the right shape for it.
- **Guardrails from section 9.3 still apply:** probe before adopting, revert automatically if the new address fails, staged rollout, https only.

Cost at 500 desktops: a 15-minute poll is 0.55 requests/second, and it lands on host C, not on the sync HQ that is already at its limits.

17.4 Three problems on host C this exposes

Making host C the anchor raises its importance sharply, and it is currently the least robust piece of the system.

1. **It has no persistent disk** (`render.yaml` declares none), so its SQLite state file lives inside the deployed code and **is wiped on every deploy**. Two consequences that exist today, before any of this work: the machines list behind `/api/updates/machines/` — your only view of which desktops exist and what version they run — resets on each deploy, and **the HQ migration lock is stored there too**. That lock is what stops two desktops migrating the shared HQ database at once. A redeploy while a fleet update is running could drop it. That is a correctness risk in the current system and worth fixing regardless of this design. Generating the endpoints document from environment variables avoids adding to the problem, because it needs no storage at all.
2. **It is on Render's free plan**, so it sleeps when idle and answers the first request slowly. As the fleet's anchor it should be on a paid always-on plan. If host C is unreachable you cannot redirect the fleet — the one thing it exists to do.

3. **Package signing is still off** (UPDATE_PUBLIC_KEY is empty, finding C-3). The same key signs the endpoints document, so this stops being a tidy-up and becomes a prerequisite: an unsigned redirect from a compromised host C would point the entire fleet wherever an attacker chose.

17.5 Until it is built

The move is still possible today, with the levers from section 15: change HQ_ENDPOINT_DEFAULTS, ship the release flagged critical, and the fleet follows within about a day. That works, and it already beats visiting 500 machines. The host C design replaces a day and a code release with a minute and an environment variable — and, more importantly, works when host A is already gone.

17.6 Order of work

1. Turn on package signing (C-3). Prerequisite for everything below.
2. Put host C on a paid plan and give it a persistent disk — or move its state to PostgreSQL. Fixes the migration lock and the machines list at the same time.
3. Add /api/endpoints/ to host C, signed, from environment variables.
4. Add the desktop client, with probe-before-adopt and automatic revert.
5. Then the admin page of section 9, as a nicer way to set what is by then just an environment variable.

18. Built: the fleet redirect, and two scaling fixes

Section 17's design, implemented, plus the cheapest capacity work. **89 tests** cover it; the full Django suite is 623 and pytest 66, all passing.

18.1 The update server now answers "where is the sync HQ?"

GET /api/endpoints/ on host C (hq_server/endpoints.py, main.py) returns a small document — the current sync address, optional database fields, fallbacks — signed with the Ed25519 key that already signs update packages. It is **built from environment variables and signed per request**, so it needs no storage and survives a redeploy intact.

Deliberately **unauthenticated**: a desktop whose sync key was lost or rejected is precisely the one that most needs to learn where HQ went, and the document carries no secrets. Integrity comes from the signature, not from who is asking. A password is structurally excluded from what it can carry.

With FLEET_SYNC_API_URL unset it reports configured: false and advertises nothing, so **silence is never read as "move to nowhere"**.

18.2 Desktops follow, but only when four things hold

endpoint_sync.py polls host C on a jittered interval, independent of the sync HQ — so a move is discoverable when host A is already gone. An address is adopted only if **all** of these pass:

1. a valid Ed25519 signature from the update server's key;
2. the document is recent, so an old one cannot be replayed to drag the fleet backwards;
3. the address passes the same validation the settings page applies;
4. the address actually answers /health.

After adoption the address is on probation. The agent's existing health check feeds note_health, and sustained failure — five consecutive failures *and* ten minutes, so a brief outage is not enough — **restores the previous**

address automatically. A bad move undoes itself without anyone on site.

Resolution is now five layers: `env > config.json (operator pin) > remote > provisioning > default`. A fleet move sits **below** an operator's deliberate pin, so a machine pointed somewhere on purpose stays there. The adopted state lives in its own `endpoints.json`, never mixed into `config.json`, and is shown on the settings page and by `hq_endpoint --show`.

Of the 29 tests on this, **11 are refusals** — unsigned, wrong key, tampered, replayed, unreachable, invalid, no key configured, unknown settings, an unconfigured server. Those are the cases that strand machines.

18.3 Host C is now fit to be the anchor

`render.yaml` gains a **persistent disk** and `HQ_STATE_DB` pointing at it, and moves off the free plan. This fixes a defect that existed before any of this work: the SQLite state lived inside the deployed code and was **wiped on every deploy**, taking with it the machine registry *and the shared-database migration lock*. Losing that lock mid-rollout lets two desktops migrate the HQ database at once. `store.py` always supported an external path; only the disk was missing.

18.4 Two scaling fixes

The retry storm is gone. HQ refuses streams beyond its cap with 503 and `Retry-After`, then the client came straight back 10 seconds later — and HQ blocks about 10 seconds before refusing, so beyond ~25 desktops every spare worker thread was busy refusing connections. The client now honours `Retry-After`, backs off exponentially to 10 minutes with jitter, and says plainly that it is falling back to interval polling. Instant push is an optimisation; correctness never depended on it.

Idle polling backs off. A desktop polled every 5 seconds whether or not anything was happening. After three quiet cycles the interval now doubles up to 120 seconds, snapping back instantly on any change or SSE wake-up, with jitter so a fleet started by the same morning routine does not arrive in lockstep.

Measured effect on idle load per desktop:

	Per desktop	At 500 desktops
Before	0.333 req/s	167 req/s
After	0.078 req/s	39 req/s

A 77% cut in the idle cost of the fleet, with no loss of responsiveness when work is actually happening.

18.5 Ratings

Moving HQ: 7.9 → 9.0 / 10. "Change reaches installed machines" is now 10: an address change needs no release at all. "Safety of a move" rises from 4 to 7 — signature, freshness, validation, probe-before-adopt and auto-revert are all in place, but the signing key still has to be generated (C-3) and client keys still do not survive a new HQ (C-4).

500 desktops: 4.2 → 5.8 / 10.

Dimension	Weight	Was	Now	Why it moved
A setting change reaching the fleet	20%	6	9	Signed remote redirect; no release needed
HQ request capacity	20%	2	4	Idle load cut 77%; still one worker
HQ database connections	20%	1	1	Untouched — the wall

Dimension	Weight	Was	Now	Why it moved
Update delivery at scale	10%	4	6	Migration lock and registry now survive deploys
Fleet visibility	10%	6	7	Machine registry persists; dashboard still unauthenticated
Safety when a change is wrong	10%	5	8	Verify, probe, auto-revert
Per-machine configuration	10%	9	9	Unchanged

The remaining gap is almost entirely one row. **Direct database connections are 20% of the weight scoring 1**, and nothing above touches them: at 500 desktops that is still 1,000–1,500 connections to Supabase. Dimension 2 is capped at 4 for the same structural reason as before — the certificate guard keeps HQ at one worker, so capacity can be reduced but not multiplied.

18.6 What is still open

1. **Generate the signing key** (`python hq_server/build_package.py --genkeys`) and set `HQ_SIGNING_PRIVATE_KEY` on host C and `UPDATE_PUBLIC_KEY` in the client build. **Until this is done the redirect refuses to work at all** — by design; it will not follow an unsigned instruction.
2. **Take the database off the desktop** (C-7, Phase 4). Now the single largest remaining item, and the only way past roughly 200 desktops.
3. **Move the certificate guard to the Redis lock** so HQ can run more than one worker.
4. **Client keys on a new HQ** (C-4) and **the unauthenticated HQ dashboard**.
5. The admin page of section 9, now just a nicer way to set an environment variable.

18.7 Unrelated defect found while linting

Not mine and not committed, but it will bite: `CalSoft/pdf_generators/header.py` calls `organisation_name()` twice without importing it. The function exists in `core/branding.py` but is never imported there, so **both call sites raise** `NameError` — and the second is inside the except handler, so the fallback crashes too. This is in the certificate PDF header path. One import line fixes it. It is in the working tree, not in HEAD, so it came from uncommitted work rather than from this task.

19. Correction: the database wall is a burst, not a ceiling

Sections 14 and 18.5 scored "HQ database connections" **1/10** on the claim that 500 desktops mean 1,000–1,500 concurrent connections to Supabase. **That claim was wrong**, and the error mattered: it made the hardest-sounding item the wrong item to work on.

19.1 What I got wrong

I counted Django's hq database alias as one persistent connection per desktop, because it sets `CONN_MAX_AGE` 300. But `Equiper/db_router.py` sends **every read and every write to default** — the local database. Nothing queries hq in normal operation; the alias exists only so `allow_migrate` can apply migrations to it. Django opens connections lazily, so a desktop that never queries hq **never connects to it at all**.

What actually connects, and how often:

Source	Cycle	Held	Average concurrent at 500 desktops
Certificate conflict guard	30 min	~3s	0.8

Source	Cycle	Held	Average concurrent at 500 desktops
Mirror	24 h	~2 min	0.7
Missing-parent recovery	on a foreign-key violation	~1s	negligible
Steady state total			≈1.5

One and a half. Not fifteen hundred. **Steady-state database load is a non-issue at 500 desktops, and would be a non-issue at 5,000.**

19.2 What is actually true

The database problem is real but it is **entirely a burst problem**, concentrated at two moments.

A fleet update. Every desktop that fails to take the migration lock runs `showmigrations --database=hq` in a subprocess (`updates/updater.py`), and that is a direct connection to the HQ database. A 100% rollout means up to 500 of them in one window. This is also exactly when the migration lock matters — and until section 18.3, that lock lived in storage wiped on every deploy. The existing `rollout_percent` already controls this: at 10% it is 50 connections, which is unremarkable.

A site powering on together. The conflict guard waited a flat 60 seconds after start before its first HQ connection, so forty desktops switched on at 8am connected in the same second. Now jittered over 60–300 seconds, turning a spike into a trickle. That is the one code change this section produced.

A complication worth knowing. `config.py` defaults `hq_db.port` to **5432**, Supabase's session mode, where each client connection holds a dedicated server connection. HQ's own `render.yaml` uses **6543**, the transaction pooler, which multiplexes and supports far more clients. For bursts, 6543 is the right choice — but **not for migrations**: Django's migration path needs session-level features that a transaction pooler does not provide. So this is not a one-line default change. The right shape is 6543 for the agent's read paths and 5432 reserved for the migration path, which is a small, deliberate piece of work rather than an edit.

19.3 Corrected rating

Dimension	Weight	\$18.5	Corrected	Why
HQ database connections	20%	1	6	≈1.5 steady-state connections; bursts are real but bounded and already controllable by rollout percentage

500 desktops: 5.8 → 6.8 / 10.

The honest summary of the correction: I measured a persistent connection that was never opened. Reading `db_router.py` — twelve lines — would have caught it at the start. The lesson is the one from section 15.3 repeated: the defect was in a seam, between a DATABASES entry that looks expensive and a router that quietly makes it free.

19.4 What now actually limits the fleet

With the database demoted, the ceiling is request capacity, and that is one structural item:

1. **HQ runs one worker** and cannot run more until the certificate re-entrancy guard moves to the Redis lock that already exists for the background thread. Idle load is now 39 req/s at 500 desktops, which one worker handles; peak activity is what will hurt.

2. **The event stream still caps at 24.** Clients now degrade gracefully to polling, so it is a lost optimisation rather than an outage — but the server still blocks about 10 seconds before refusing, which wastes a thread per refusal. The server-side fix is to refuse immediately unless this client just evicted its own stream.
3. **Update bandwidth** at 500 desktops has never been measured.

None of these is a hard ceiling. All three are ordinary capacity work.

20. Where each kind of change is actually made

Three different changes, three different workflows. Confusing them is the main way this goes wrong.

20.1 Moving the sync HQ from host A to host B

Not on your laptop. No git, no rebuild.

Change one environment variable on **host C** in the Render dashboard:

```
FLEET_SYNC_API_URL = https://host-b.example.com/api/sync
FLEET_SYNC_FALLBACKS = https://host-a.example.com/api/sync    (during the move)
```

Restart the service. Every desktop picks it up within `FLEET_POLL_SECONDS` (15 minutes by default), verifies the signature, tests that host B answers, and switches. Host A can be switched off once the fleet has moved.

That is the entire operation, and it is the point of sections 17 and 18. **Keep host A in the fallback list until you are satisfied**, so a desktop that was switched off during the move is not stranded.

20.2 Changing the desktop application

Laptop → GitHub → Render rebuilds → desktops download. This is the flow you described, and it is correct for code.

Both the desktop app and host C live in the **same repository**, and host C packages the repository root (`repo_root=BASE_DIR.parent`). So one push does two things: it redeploys host C, and host C then builds the desktop update package **from that same commit**.

The trigger is `hq_server/version.txt`. On startup host C builds a package for whatever version that file names, and skips if a package for that version already exists. **Push without bumping it and nothing reaches the fleet** — the code deploys, no package is built, desktops see no update.

So the sequence is:

1. Make the change on your laptop.
2. Bump the version in **three places that must agree**: `version.txt`, `hq_server/version.txt`, and `APP_VERSION` in `Equiper/settings.py`.
3. Push to GitHub. Render redeploys host C, which builds the package on startup.
4. Desktops check `/api/updates/latest/` and download. Flag the release `critical` if you want it applied without someone approving it on each machine.

Use `rollout_percent` to stage it. That also bounds the migration burst from section 19.2.

20.3 Changing host C itself

Same push as 20.2 — it is the same repository and Render auto-deploys `hq_server/` on push. Only the environment variables are set in the dashboard rather than in the file, which is why `render.yaml` marks the secret

ones sync: false.

20.4 Moving the sync HQ's own code

~/Desktop/hq_server is a **separate repository with its own Render service**. Standing up host B means deploying that repository to a new service and pointing it at the same database. Nothing in this document changes that; section 20.1 is only how the *fleet finds out*.

20.5 Two more fixes this question surfaced

Built packages were also on ephemeral storage. PACKAGES_DIR was inside the deployed code, so a redeploy wiped every *previous* version's package and manifest. build_delta_zip needs the old manifest to work out what changed — without it, every desktop falls back to a **full download**. At 500 desktops that is the difference between a few megabytes each and the entire package each, and it was silently happening after every deploy. Packages now live on the persistent disk (HQ_PACKAGES_DIR).

APP_VERSION in render.yaml **said 1.0.0** while every other version file said 1.5.0. Harmless today, because _read_version() prefers version.txt, but it is exactly the kind of drift that makes a release look deployed when it is not. Corrected, with a comment saying which file actually decides.

20.6 The order for the first time

The redirect cannot deliver itself. Once, and in this order:

1. python hq_server/build_package.py --genkeys. Put the private half in host C's HQ_SIGNING_PRIVATE_KEY, the public half in the client as UPDATE_PUBLIC_KEY. **Until this is done the redirect refuses to work**, by design.
2. Add the disk to host C and move it off the free plan (render.yaml, already written).
3. Bump the three version files, push. Host C redeploys with /api/endpoints/, and builds the desktop package containing the client.
4. Publish that release flagged critical, staged by rollout_percent. Watch /api/updates/machines/ — which now survives deploys — until the fleet is on it.
5. From then on, moving HQ is section 20.1: one environment variable.

21. Closing the two safety gaps (C-3, C-4)

21.1 The signing key could not actually be turned on

C-3 was recorded as "signing is off". It was worse than that: **there was no supported way to turn it on**.

updates/updater.py reads the trust anchor from settings.UPDATE_SYSTEM["public_key"], but settings.py never set that key and config.py had no update.public_key. The only remaining path was hand-editing the UPDATE_PUBLIC_KEY = "" constant in updater.py — a source change, in every build, for a value that is not even secret. That is why it had stayed empty.

Now wired end to end: update.public_key in the config defaults, overridable by UPDATE_PUBLIC_KEY, exported to the environment, and passed into UPDATE_SYSTEM.validate_config() reports plainly when it is empty, saying that packages are applied unverified and address changes cannot be followed. It is deliberately **not** something HQ can set remotely — usable_endpoints only accepts keys that exist in HQ_ENDPOINT_DEFAULTS, so a server cannot nominate the key used to verify it.

Proven end to end against a freshly generated keypair: host C signs a move to host B, a desktop on the shipped default verifies it, probes it, adopts it (source: remote), and the same document offered under a different key is

refused with "signature does not match the update server's key".

21.2 A move would have broken every client's key, silently

C-4 said client keys do not survive a new HQ. Checking how that failure would actually present: **the agent has no handling for 401 or 403 anywhere**. A rejected key surfaced as `Download` returned status 401 in a debug line, repeated every few seconds, with nothing connecting it to the move that caused it. The fleet would stop syncing and the logs would not say why.

Two changes:

- The agent now recognises a rejected key on the download and heartbeat paths and logs one clear message every ten minutes: that HQ refused this client's key, that sync will not work until it is accepted, that **if HQ was moved its sync_api_keys table must move with it**, how to re-issue a key otherwise, and that local work is unaffected and will upload once the key is valid. If the machine recently followed an address change it names that move and when it happened.
- `SECURITY.md` gains a **Moving HQ to a new host** runbook. The essential point is that addresses take care of themselves and keys do not: copy `sync_api_keys` to the new host *before* announcing the move. The values are hashes bound to a `client_id`, so the tokens desktops already hold keep working. Re-issuing keys individually is the fallback, and at 500 desktops it is 500 admin calls.

This is deliberately not a new authentication flow. The endpoint document is served unauthenticated so that a key-less desktop can still find HQ, which means it must never carry a credential; and inventing a cross-service trust path between the update server and a new sync host would add more risk than it removes. Copying one table is the smaller, safer answer — it just has to be written down, and the failure has to be legible when someone forgets.

21.3 Ratings

Dimension	Weight	Was	Now
Safety of a host move (moving-HQ rating)	10%	7	9
Safety when a change is wrong (500-desktop rating)	10%	8	9

Moving HQ: 9.0 → 9.2 / 10. Signing is now switchable and verified end to end, and the key failure is legible. Not 10: the key ceremony is still a manual step someone has to perform, and nothing yet checks that host B carries the keys *before* the move is announced.

500 desktops: 6.8 → 6.9 / 10. Barely moved, correctly — these were correctness and operability gaps, not capacity. The ceiling is still HQ's single worker.

21.4 Remaining, in order

- Perform the key ceremony.** `python hq_server/build_package.py --genkeys`, private half to host C, public half into the build. Everything signed depends on it and nothing else can be done first.
- Move the certificate guard to the Redis lock** so HQ can run more than one worker. This is now the capacity ceiling.
- Refuse SSE immediately** instead of blocking ten seconds, so a refusal costs no thread.
- Authenticate the HQ dashboard**, which today exposes every client's name, IP and version to anyone.
- Take the remaining direct database paths off the desktop — no longer urgent after section 19, but it is what removes the burst entirely.

22. The four remaining items

22.1 The key ceremony — not done, deliberately

I did not generate your signing key, and I should not. The private half would land in this transcript and in tool logs, and whoever holds it can sign packages that every desktop executes. That is remote code execution on the whole fleet from a string sitting in a chat history. A production signing key should be generated where it will live, by someone who can put it straight into the secret store.

What I did instead is make the ceremony safe to perform and easy to verify:

- `build_package.py --genkeys` now prints instructions, not just two blobs: exactly where each half goes, that the private half must never reach a chat, ticket or log, that the public half is not secret and belongs in the build, and that losing the private half means re-keying every client.
- `hq_endpoint --show` now reports **Signing: ON** or **Signing: OFF** with the consequence spelled out, so you can confirm the ceremony worked from any desktop.

A bug this turned up. `UPDATE_PUBLIC_KEY` worked in development and was **silently ignored in the packaged app** — the same failure as C-2, which I had fixed for addresses and not noticed here. There is now a `PLAIN_ENV` layer for non-secret settings that must be overridable in production too, and the value is never written to `config.json`, so unsetting the variable cannot leave a stale key behind. Found by running the command and seeing "Signing: OFF" when it should have said ON.

A second bug, in my own fix: the first-run save persisted the environment value anyway, because I stripped it in only one of the two save paths. Both paths now share one implementation.

22.2 The certificate guard — already done, and the Procfile is out of date

This was on the list as "move the certificate guard to the Redis lock so HQ can run more than one worker".

Reading the code, that work has already been done. The constraint is now the documentation, not the code:

Path	Protection	Cross-process safe?
Number allocation (<code>get_next_number</code>)	<code>pg_advisory_xact_lock</code> , held to commit	Yes
Per-session allocation (<code>certificate_service_extra</code>)	<code>SELECT ... FOR UPDATE</code> on the session row	Yes
Batch run, background thread	Redis <code>cert_processor_lock</code>	Yes
Batch run, HTTP <code>/generate_certificates</code>	The same Redis key	Yes
In-memory <code>self.processing</code>	Within one process only	No — but it only prevents duplicate work

The dangerous outcomes — two numbers for one session, or one number for two sessions — are prevented at the **database** level, which does not care how many workers there are. The comment on the `FOR UPDATE` names precisely the race it closes. The in-memory guard and the Redis lock are efficiency measures layered on top.

So `--workers 1` in the Procfile and `render.yaml` looks like it outlived the reason for it.

I have not changed it, for three reasons. It is the most consequential setting in the system and a wrong call produces duplicate certificate numbers, which is a real-world problem rather than a software one. The evidence is from reading code, not from a concurrency test. And there is uncommitted work in the HQ repository right now — `test_certificate_number_protection.py` — on exactly this area, so the person writing it has context I do not.

Recommended instead: raise workers to **2** (not 32) on a staging instance, run the existing certificate tests against a real PostgreSQL — 53 of the 62 skip without one — add a test that drives two workers at the same pending session, and watch for gaps or duplicates. If that holds, raise it further. This is the single highest-value capacity change available, and it deserves an hour of deliberate testing rather than a config edit.

22.3 SSE now refuses immediately

`sync_stream` waited up to ten seconds for a slot before answering 503. Beyond the cap each surplus desktop occupied a worker thread for ten seconds purely to be told no, then came back — with 32 threads and a cap of 24, the few spare threads were permanently consumed refusing connections and the server went deaf to everything, health checks included.

The wait had a real purpose, so it is kept where it earns its place: a device reconnecting evicts its own previous stream, and that stream needs a moment to release. So the rule is now **wait only when reclaiming your own slot; refuse everyone else instantly**. Combined with the client-side backoff from section 18.4, the stream cap degrades into "some desktops poll instead of being pushed to" rather than an outage.

22.4 The HQ dashboard is no longer open to the world

`/api/sync/dashboard` renders HTML listing every client's name, IP address, version and workshop data, and had **no authentication of any kind**.

It now requires `HQ_ADMIN_TOKEN`, accepted two ways because it is a page as well as an endpoint: `Authorization: Bearer ...` for scripts, or HTTP Basic with the token as the password so a browser can prompt and open it. It **fails closed** — with the variable unset nothing is reachable, because for a route exposing this data "unconfigured" must never mean "open".

The same guard replaces the two SSE admin endpoints' checks against `SYNC_AUTH_TOKEN` — the shared token every client carries, which meant any client could list and evict other clients' streams. The key-admin routes had already moved to `HQ_ADMIN_TOKEN`; these two were missed.

Verified directly: unset token → 503; no credentials, wrong bearer, wrong basic → 401 with a `www-Authenticate` challenge; correct bearer and correct basic → 200.

22.5 Where this leaves the ratings

	Before	After
Moving HQ	9.2	9.4
500 desktops	6.9	7.4

Fleet visibility rises (the dashboard is no longer a data leak), safety rises (signing is switchable and verifiable), and HQ request capacity rises from 4 to 6 — the stream no longer burns threads refusing connections. It is not higher because **the one worker remains**, and that is now the entire ceiling. If 22.2 is confirmed on staging, HQ request capacity goes to 8 or 9 and the overall figure to roughly 8.5, which would make 500 desktops comfortable rather than merely possible.

22.6 Caveats on this round

- The HQ suite runs **9 tests and skips 53** without PostgreSQL, so my changes there are covered by direct behavioural checks rather than by that suite. Running it against a real database is worth doing before deploying.

- Changes in ~/Desktop/hq_server sit alongside someone else's uncommitted work on a feature branch. I touched four files — `security_layer.py`, `queue_dashboard.py`, `job_queue_download_update.py`, `og_server_core.py` — and none of the four they had already modified.

